TEMA 2. Estructuras de control



Desarrollo Web en Entorno Cliente.

Índice :

- Estructuras de control selectivas
  - If - else
  - switch
- Estructuras de control iterativas (bucles/loop)
  - Bucle while
  - Bucle for. (for/in , for /of)
  - Bucle do-while
- Bibliografía.

Estructuras de control

Las estructuras de control de flujo son los bloques de construcción que permiten modificar el orden en el que se ejecutan las instrucciones de un programa. Por defecto, un programa lee y ejecuta el código línea por línea de arriba a abajo (de manera secuencial). Con las estructuras de control, puedes tomar caminos opcionales, repetir bloques de código o romper secuencias. Se dividen principalmente en **tres tipos**:

- Estructuras Secuenciales
- Estructuras Selectivas (o Condicionales)
- Estructuras Iterativas (o Repetitivas / Bucles)

Estructuras Secuenciales

Es la forma por defecto en la que se ejecuta el código: cada orden o sentencia se ejecuta una detrás de otra en el orden en que fueron escritas.

Ejemplo:

```
let nombre = "Ana";
let saludo = "Hola, " + nombre;
console.log(saludo); // Se ejecuta estrictamente en orden
```

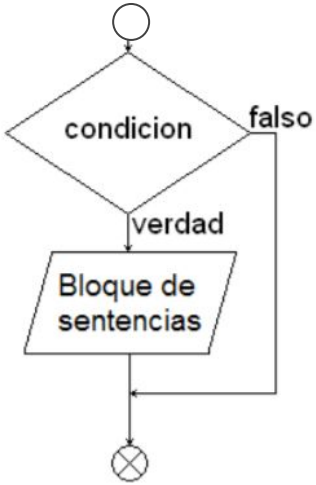
Estructuras Selectivas (o Condicionales)

Permiten evaluar una o varias condiciones Booleanas (true o false) y decidir qué bloque de código ejecutar en función del resultado. Existen 3 tipos:

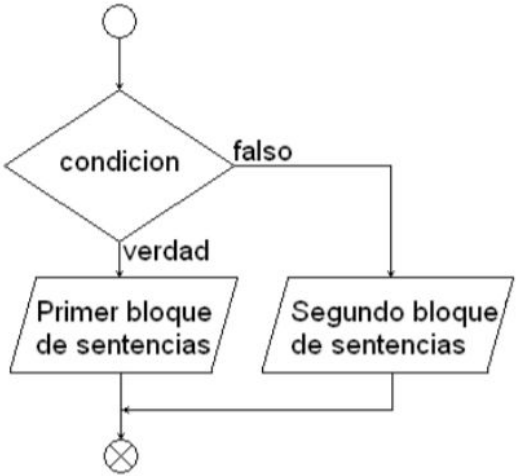
- **Simple (if):** Ejecuta un bloque si la condición es verdadera.
- **Doble (if...else):** Ejecuta un bloque si es verdadera y otro distinto si es falsa.
- **Múltiple (if...else if...else o switch):** Permite evaluar varias condiciones consecutivas o elegir entre múltiples opciones posibles según el valor de una variable.

Ejemplos con su diagrama de flujo:

```
if (condicion) {
    Bloque de sentencias
}
```



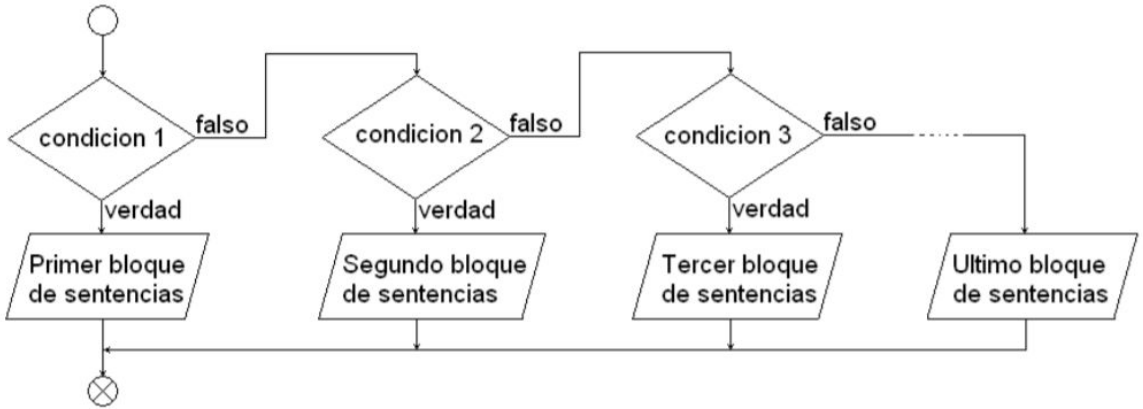

```
var n1=5;
var n2=4;
if (n1 > n2){
  alert ("Mayor");
} else {
  alert ("menor");
} //Muestra "Mayor"
```




```
if (condicion) {
    Primer bloque de sentencias
}else{
    Segundo bloque de sentencias
}
```

```
if (condicion 1) {
    Primer bloque de sentencias
}else if (condicion 2) {
    Segundo bloque de sentencias
}else if (condicion 3) {
    Tercer bloque de sentencias

}else{
    Ultimo bloque de sentencias
}
```



switch

El **switch** en JavaScript evalúa una expresión y ejecuta el bloque de código cuyo valor coincida con dicha expresión (mediante una **comparación estricta**, equivalente a ===). Es una alternativa más limpia a tener múltiples declaraciones if...else if.

La estructura básica es la siguiente:

```
switch (expresion) {
  case valor1:
    // Código si expresion === valor1
    break;
  case valor2:
    // Código si expresion === valor2
    break;
  default:
    // Código si no coincide con ningún caso previo
    break;
}
```

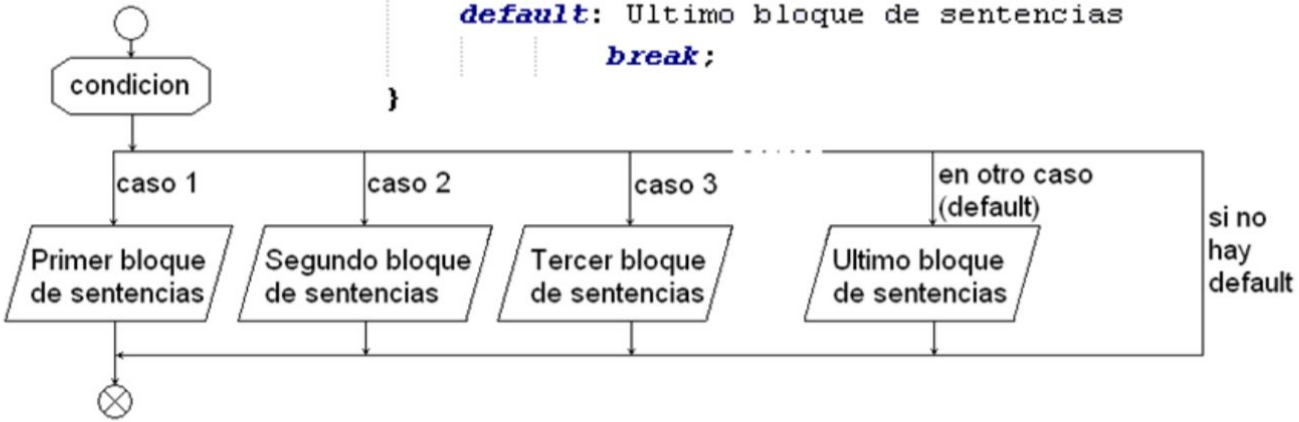
switch

Componentes clave:

- **switch (expresion)**: Evalúa el valor inicial solo una vez.
- **case valor:**: Compara el resultado de la expresión contra el valor. Debe coincidir tanto en tipo como en contenido (por ejemplo, '5' no es igual a 5).
- **break**: Detiene la ejecución del switch. Si se omite, el código continuará ejecutando los siguientes casos automáticamente (*fall-through*), sin importar si coinciden o no.
- **default**: Opcional. Se ejecuta únicamente si ningún case coincidió. Funciona como el else final.

```
switch (expresion) {
    case 1: Primer bloque de sentencias
            break;
    case 2: Segundo bloque de sentencias
            break;
    case 3: Tercer bloque de sentencias
            break;

    default: Ultimo bloque de sentencias
            break;
}
```




```
var n1=3;
switch (n1) {
   case 1:
         alert ("valor 1");
         break;
   case 2:
         alert ("valor 2");
         break;
   case 3:
          alert ("valor 3");
          break;
   default:
          alert("defecto");
          break;
} // Muestra "Valor 3"
```

Ejemplos de casos agrupados:

```
let dia = "Lunes";

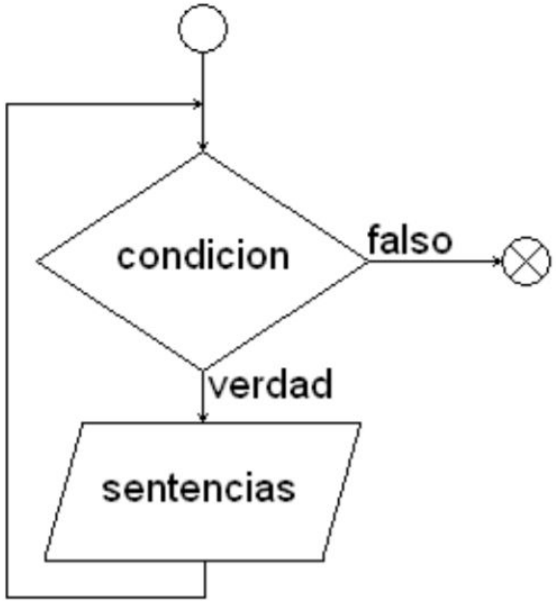
switch (dia) {
  case "Lunes":
    console.log("Inicio de semana.");
    break;
  case "Viernes":
    console.log("Casi el fin de semana.");
    break;
  case "Sábado":
  case "Domingo": // Casos agrupados
    console.log("Es fin de semana.");
    break;
  default:
    console.log("Día laboral intermedio.");
    break;
}
```

```
let fruta = "manzana";

switch (fruta) {
  case "manzana":
  case "pera":
  case "plátano":
    console.log("Es una fruta común.");
    break;
  default:
    console.log("Fruta desconocida.");
}
```

Bucle iterativo while

```
while (condicion) {
    Sentencias
}
```

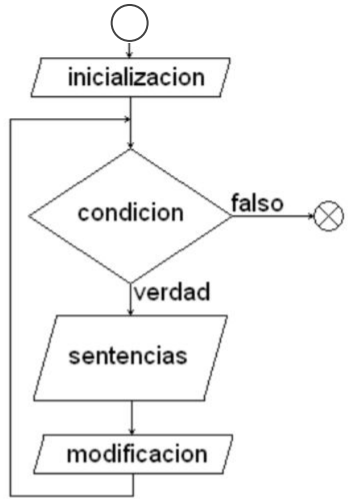



```
var x=1;
while (x<=5){
    alert ("Valor de X : " + x);
    x++;
}
```

Bucle iterativo for

Es un bucle iterativo que se ejecuta un número determinado de veces, aunque estas pueden ser variables. Tiene tres secciones: inicialización, condición y modificación.

```
for (iniciacion ; condicion
                ; modificacion) {
    sentencias
}
```




```
var x;
for ( x = 1; x <= 5; x++ ){
    alert ("Valor de X : " + x);
}
```

for / in

Recorre las propiedades de un objeto.

```
<!DOCTYPE html>
<html>
<body>
<p>Haz click para recorrer las propiedades del objeto.</p>
<button onclick="myFunction()">Click</button>
<p id="demo"></p>
<script>
function myFunction() {
  var persona = {nombre:"John", apellido:"Doe", edad:39};
  var text = "";
  var x;
  for (x in persona) {
    text += persona[x] + " ";
  }
  document.getElementById("demo").innerHTML = text;
}
</script>
</body>
</html>
```

for / of

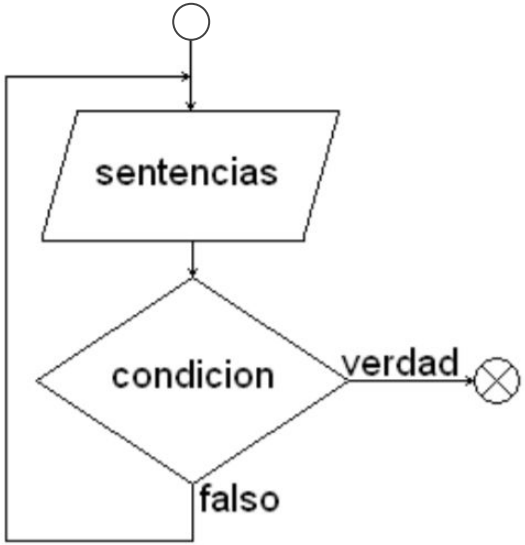
Recorre los elementos de un objeto iterable.

(ej: un array).

```
<!DOCTYPE html>
<html>
<body>
<script>
var coches = ['BMW', 'Volvo', 'Mini'];
var x;
for (x of coches) {
  document.write(x + "<br>");
}
</script>
</body>
</html>
```

Bucle iterativo do-while

```
do {
    sentencias
} while (condicion);
```




```
var x = 1;
do {
    alert ("Valor de X : " + x);
    x++;
} while (x <= 5);
```

Bibliografía

- LibrosWeb
- W3Schools
- Developer Mozilla Docs